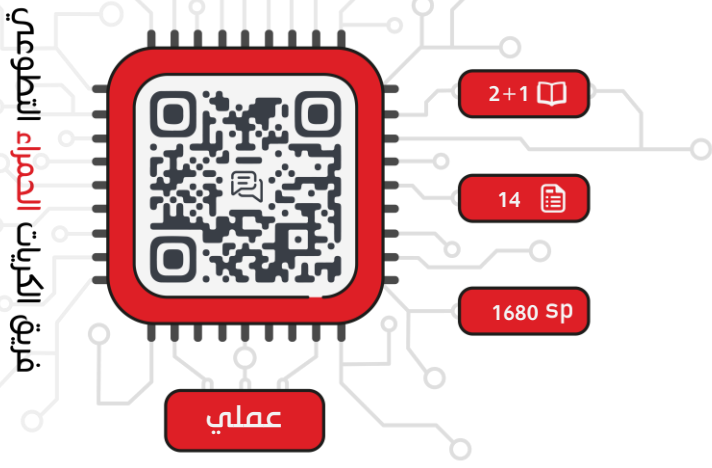


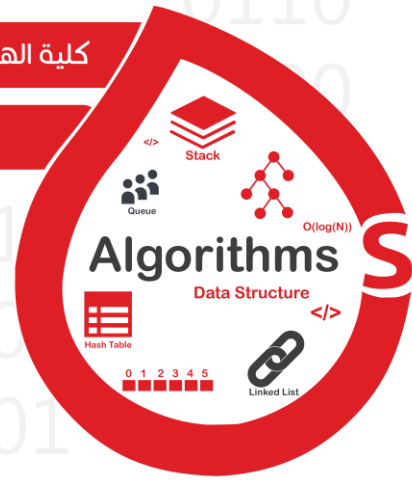


كلية الهندسة المعلوماتية

السنة الثانية

تعقيد الخوارزمية

عملي مشترك



RB Informatcs; 29/10/2024

خوارزميات 1

السلام عليكم ورحمة الله وبركاته

أهلاً بكم يا أصدقائي في مقرر الخوارزميات وبنى المعطيات ، سنكون سوياً في القسم العملي من هذا المقرر الممتع :

ماذا سنأخذ في هذه المحاضرة ؟

الخوارزمية وخصائصها - التعقيد وأنواع - أمثلة على حساب التعقيد - مقدمة في العودية .

الخوارزمية وخصائصها

تعريف الخوارزمية:

هي مجموعة من الخطوات المحددة المتسلسلة المنطقية المنتهية التي تهدف لحل مشكلة ما (سواء كانت المشكلة هندسية-رياضية-حياتية...) ولا وجود لخوارزمية دون وجود مشكلة تسعى إلى حلها.

خصائص الخوارزمية:

1. منتهية :

أي أنه لا توجد خوارزمية غير منتهية (مثلاً تحوي على حلقات غير منتهية) لماذا؟

لأننا -بكتابتنا للخوارزمية- نبحث عن حل نهائي لمشكلة معينة، ووجود خوارزمية لا تنتهي بحل معين لا يفيدنا في حل المشكلة المطلوبة.

2. صحيحة وواضحة:

أي أن خطوات الخوارزمية واضحة وغير مبهمه ،ليست مكتوبة بشكل سردي كي لا تتحمل عدة أوجه للتفسير.

3. لها دخل (Input) وخرج (Output):

أي يجب أن يكون لها خرج ، وهو حل المشكلة المرادة ، ومن الممكن أن يكون للخوارزمية أكثر من خرج .
أما بالنسبة للدخل فيوجد لأغلب الخوارزميات دخل يمكن أن يكون 0 أو أي شيء آخر ، و يمكن أن تتواجد خوارزمية بدون دخل، مثل : خوارزمية تطبع hello world ليس لها دخل ولكن لها خرج.

4. فعالة :

أي أنها تؤدي وظيفتها بشكل تام وهي حل المشكلة المطلوبة.

5. قابلة للبرهان:

أي أنه يمكن برهنة الخوارزمية بأي طريقة (الاستقراء الرياضي-نقض الفرض -...)

6. تمتلك لغة خاصة :

أي أن لغة كتابة الخوارزمية مستقلة عن لغة البرمجة التي ستنفذ بها (C++, Java, Python,...) ، ف لغة كتابة الخوارزمية تعرض خطوات الحل والتي يمكن تحويلها فيما بعد إلى برنامج مكتوب بأي لغة برمجة كانت.

بعض طرق كتابة الخوارزمية

⚡ مخططات flowchart :

هذه المخططات عبارة عن مجموعة من الأشكال حيث أن كل شكل يمثل تعليمة معينة، يتم الوصل بين هذه الأشكال بأسهم حتى نحصل على الخوارزمية كاملة خطوة تلو الأخرى. "لن نستخدم هذه الطريقة في مقررنا"

⚡ اللغة البسيطة Pseudocode :

وهي عبارة عن كتابة خطوات الخوارزمية بلغة بسيطة دون الدخول بالتفاصيل .
وهنا لا نقصد الكتابة بلغة برمجة معينة بل بلغة عامة، حيث نستطيع تحويل الـ Pseudocode إلى أي لغة برمجة لاحقاً.

مثال أرييسزي بسيط : يمكن كتابة : `for ( int i = 0 ; i < n ; i ++ )` بالشكل : `for ( i → n )`

أمثلة عن كتابة بعض الخوارزميات بال pseudocode

✍ مصفوفة أحادية فيها n عنصر ، نريد طباعة كل عنصر فيها إن كان زوجي أو فردي :

```

1  for( int i = 0 ; i < n ; i++ )
2      {
3          if(a[i]%2 == 0)
4              cout<<"even\n";
5          else
6              cout<<"odd\n";
7      }

```

← حلقة for تمر على جميع عناصر المصفوفة $a[n]$

شرح أريسيزي :

- نلاحظ هنا أنه لم نعرف المتغيرات n و $A[]$ ، لأنه بمجرد النظر إلى الكود سيفهم معناهم، لكن تعريفهم ليس خطأ أيضاً.
- نلاحظ أنه عند الطباعة لا نهتم كثيراً بشكل الطباعة (endl ، فراغات ، أحرف كبيرة، ..) بل نهتم بأن تعطي الطباعة الغاية المرادة.

✍ إيجاد max و min في مصفوفة :

min	max
<pre> 1 int min = A[0]; 2 for(int i=1;i<n;i++) 3 if(A[i]<min) 4 min=A[i]; </pre>	<pre> 1 int max = A[0]; 2 for(int i=1;i<n;i++) 3 if(A[i]>max) 4 max=A[i]; </pre>

✍ بحث عن عنصر معين في المصفوفة وطباعته :

```

1  int num;
2  for(int i=0 ; i<n ; i++)
3      {
4          if(A[i] == num)
5              cout<<"found!";
6      }

```

← n هو حجم المصفوفة A وضوحاً

ملاحظة أرييسيزية:

ال pseudocode لا يهتم بالتفاصيل البديهية أو ال syntax بشكل صارم " ليس له قواعد ثابتة " ولكن يشترط أن يكون واضحاً.

↪ ترتيب مصفوفة عناصرها n :

يوجد عدة خوارزميات لترتيب المصفوفة ، منها طريقة الفرز الفقاعي (bubble sort) ، وهي:

شرح أرييسيزي عن ال bubble sort :

↪ هي خوارزمية تستخدم for داخل for ، حيث آلية العمل :

```
1 for( int i=0 ; i<n-1 ; i++)
2   for( int j=0 ; j<n-1 ; j++)
3     if(A[j]<A[j+1])
4       swap(A[j],A[j+ 1]);
```

• ال for الداخلية :تقوم بمقارنة كل عنصر $A[j]$ مع

العنصر الذي يليه $A[j+1]$ ، ثم تقوم بـ swap()

للتبديل بين العنصرين إذا تحقق أحد الشرطين:

← $A[j] > A[j+1]$ للترتيب التصاعدي

← $A[j] < A[j+1]$ للترتيب التنازلي

عند انتهاء هذه الحلقة ستكون $A[j]$ بمكانها الصحيح.

• ال for الخارجية : تقوم بتكرار ال for الداخلية حتى يصبح كل عنصر من عناصر المصفوفة في مكانه المطلوب ، والشرط

يكون $i < n-1$ لأن العنصر الأخير سيكون موجود في مكانه ولا داعي للمرور عليه.

إثراء : يوجد عدة طرق لترتيب المصفوفة ، منها :

Merge sort , quick sort , selection sort , insertion sort ,

فائدة : يمكن أن نختار أي طريقة لحل المشكلة المطلوبة بما أن الخوارزمية المختارة منتهية وفعالة.

لكن حتى تكون مبرمج وحش .. يجب أن تختار أفضل طريقة ، طب كيف؟

من خلال اختيار الخوارزمية ذات أفضل تعقيد.

التعقيد

↪ أنواع التعقيد:

1. التعقيد الذاكري (Space Complexity):

هو عبارة عن المساحة التي تستهلكها الخوارزمية أو الكود عند تنفيذه.

• **مثال:** التعقيد الذاكري للتعليمة: `int i;` هو 4 byte

• **تنويه (!):**

الذاكر في عصرنا الحالي تصبح أكبر وأكبر ، فالـ space complexity لن نركز عليه في مقررنا، فنحن سنهتم بالـ time complexity .

2. التعقيد الزمني (Time Complexity):

هو الزمن اللازم لتنفيذ التعليمات في الخوارزمية.

• **شرح:**

زمن تنفيذ الخوارزمية ليس له واحدة لأنه يعتمد على مواصفات الحاسب الذي يتم تنفيذ الكود عليه (حجم الـ RAM - سرعة المعالج-...) حيث لكل حاسب زمن معين لتنفيذ التعليمة الواحدة ، **كيف تقاس سرعة الخوارزمية إذاً؟**

تقاس سرعة الخوارزمية بمقارنتها بخوارزميات أخرى ، وهذه المقارنة تعتمد على:

1. أن تنفذ الخوارزمية عدد كبير من العمليات خلال زمن قليل ، أي أن تكون الخوارزمية ذات أداء عالٍ.
2. أن تكون الخوارزمية قابلة للتطوير .

• **مثال :**

بفرض لدينا : خوارزمية -1- تقوم بـ 100 عملية خلال 10s / خوارزمية -2- تقوم بـ 1000 عملية خلال 5s . فأَي خوارزمية أفضل هنا؟

" الخوارزمية الأفضل هي الأسرع عند دخل أكبر - أي الخوارزمية -2- ☺ "

▪ ترتيب التعقيد من الأفضل إلى الأسوأ:

الأفضل (الأصغر)

الأسوأ (الأكبر)

$O(1) \rightarrow O(\log_k n) \rightarrow O(\sqrt{n}) \rightarrow O(n) \rightarrow O(n \log_k n) \rightarrow O(n^k) \rightarrow O(2^n) \rightarrow O(n!) \rightarrow O(n^n)$

حساب التعقيد الزمني

كيف يتم حساب التعقيد الزمني لخوارزمية؟

- يتم حساب عدد العمليات التي يتم تنفيذها خلال الخوارزمية ، ويوجد نوعين من العمليات :
عمليات غير مكلفة : (الإسناد ، التعريف ، التصريح بتابع ، الطباعة) .

قد تكون كلفة العمليات غير المكلفة معدومة أو صغيرة جداً فنهملها عند حساب التعقيد الكلي، لكن لا نقصد بـ "غير مكلفة" أن جميع هذه العمليات كلفتها (0).

عمليات مكلفة: (حلقات for و while - العمليات الحسابية المعقدة - الشروط if و if-else و switch ... حسب محتواها).

- والآن يا أصدقائي سنقوم بشرح كيف نسحب تعقيد الخوارزمية شيئاً فشيئاً مع الأمثلة التالية:

تمرين :

العملية	الكلفة
تعريف دالة	0
تعريف متغيرات	0
إسناد	1
إسناد	1
إسناد	1
إرجاع قيمة	1

- الآن سنقوم بحساب التعقيد:

- نلاحظ أنه لا يوجد عمليات متداخلة: $T(n) = 1+1+1+1=4 \rightarrow O(1)$

ملاحظات:

- عملية التعريف غير مكلفة أبداً ، أي أن كلفتها (0) "سواء تعريف متابع أو متغيرات".
- عمليتا الإسناد والإرجاع كلفتها قليلة جداً وهي (1) .
- إذا كانت التعليمات غير متداخلة (أي لا يوجد حلقة داخل حلقة) عندها نقوم بجمع التعقيدات.
- نتجاهل الثوابت في التعقيد ، أي إذا كان الناتج عبارة عن رقم فقط هذا يعني أن عدد العمليات في الخوارزمية معروف، فيكون التعقيد عندها $O(1)$ مهما كان الرقم الناتج.

ملاحظة أريسيزية:

$T(n)$: هي تابع التعقيد بكل تفاصيله / $O(n)$: هي التعقيد النهائي بأسوأ حالة بعد إهمال الثوابت

تمرين:

```
1 int a=0;
2 for(int i=0 ; i<n ; i++){
3     a=a+1;
4 }
```

العملية

الكلفة

تعريف+إسناد

1

حلقة مكررة n مرة

n

• الآن سنقوم بحساب التعقيد:

- نلاحظ أنه لا يوجد عمليات متداخلة: $T(n) = 1+n \rightarrow O(n)$

ملاحظات:

- حلقة الـ for التي يزداد عددها بمقدار 1 كل دورة (تكرر n مرة) تعقيدها $O(n)$.
- في هذا المثال نتجاهل الثوابت فيبقى $O(n)$ "أما في المثال السابق لم يكن هناك متغير لذا كان التعقيد $O(1)$ ".

ملاحظة:

- بفرض كان الناتج: $O(n) \leftarrow T(n) = 100n$ / $O(n) \leftarrow T(n) = 10n+5$ "الأمثال والثوابت مهمة عند حساب التعقيد".

تمرين:

```
1 int f(int i) ;
2 int j=0;
3 int k,n;
4 for(int i=0 ; i<n; i++)
5     { .....}
6 return 0 ;
```

تصريح بتابع

0

تعريف+إسناد

1

تعريف

0

حلقة مكررة n مرة

n

إرجاع قيمة

1

• الآن سنقوم بحساب التعقيد:

- نلاحظ أنه لا يوجد عمليات متداخلة: $1+n+1 = 2+n$ نهمل الثوابت $O(n)$.

ملاحظات:

- عملية التصريح بتابع غير مكلفة أبداً، أي أن كلفتها $O(0)$.

تمرين أريسيزي :

```

1  int f(int i){
2      int j=0;
3      int k,n;
4      n=10;
5      for(int i=0 ; i<n; i++)
6          { .....}
7          return 0 ;
8  }

```

• سنأخذ المثال السابق ونضيف عليه السطر 4 :

- **نحسب التعقيد** : $O(n) \leftarrow T(n) = 1+1+n+1 = 3+n$.

لكن : نلاحظ هنا أن $n=10$ ، أي أن n ثابتة ومحددة ومنه:

$$O(n) = O(10) = O(1)$$

"نعود إلى فكرة أنه إذا كان عدد العمليات معلوم (لا يوجد

مجهول في المعادلة) $\leftarrow O(1)$

تمرين :

```

1  for(int i=0 ; i<n ; i+=3)
2  { .... }

```

- نلاحظ أن i تزداد بمقدار 3 كل دورة ،

أي بفرض $n=12 \leftarrow i = 0,3,6,9,12$

بالتالي عدد مرات التكرار: $T(n) = \frac{n}{3}$ بالتالي $O(n)$

تمرين :

```

1  for(int i=0 ; i<n ; i*=2)
2  { .... }

```

- نلاحظ أن $i=0$ بالتالي لن تزداد ولن تصل إلى n ،

وبالتالي : $T(n) = O(\infty)$

تمرين :

```

1  while (i<n)
2  {
3      i++;
4  }

```

- نلاحظ أنه في حلقة الـ while يتم تغيير قيمة العداد داخل الحلقة، لذا يجب الانتباه إلى محتوى الحلقة دائماً.

- العداد يزداد بمقدار 1 كل دورة ، أي أن الحلقة ستتكرر n مرة $\leftarrow O(n)$.

تمرين :

ملاحظة:

```

1 while (i<n)
2 {
3   i*=k;
4 }

```

- أي حلقة يتم التغيير على العداد الخاص بها بالضرب أو القسمة بمقدار معين k ، يكون تعقيدها $O(\log_k(n))$ ويمكن أن يكون الـ k عدداً.
- بالتالي تعقيد الحلقة السابقة : $O(\log_k n)$.

شرح عن $\log_k(n)$:• التعقيد: $O(\log_2 n)$ ، لماذا؟

```

1 for(int i=1 ; i<n ; i*=2){
2   .....
3 }

```

- لأن $(\log_k n)$ يعبر عن عدد المرات التي ستضرب k بنفسها (بفرض $i = 1$) حتى تصل إلى n .

مثال بسيط:

بفرض : $n = 32 \leftarrow \log_2(32) = 5$ و (5) هو عدد مرات تكرار الحلقة حتى تصبح $i = n = 32$.

تمرين :

```

1 for( int i=0 ; i<n ; i++ ){
2   for( int j=0 ; j<m ; j++ ){
3     .....
4   }
5 }

```

→ حلقة تتكرر n مرة → n
 → حلقة تتكرر m مرة → m

• نحسب التعقيد:

نلاحظ أن الحلقتين متداخلتين : $n*m \leftarrow O(n*m)$.

ملاحظة:

إذا كانت العمليات متداخلة (يوجد حلقة داخل حلقة) نقوم بضرب التعقيدات.

تمرين:

1 int a,b;	تعريف	0
2 a=b=0	إسناد	1
3 for(i=0 ; i<n ; i++){	حلقة تتكرر n مرة	n
4 while(j<n){	حلقة تتكرر $\log_3 n$ مرة	$\log_3 n$
5 j=j*3;}		
6 }		
7 for(k --> n)	حلقة تتكرر n مرة	n
8 { ... }		

• نحسب التعقيد:

-ننتبه إلى الحلقات المتداخلة والتعليمات غير المتداخلة:

$$T(n) = 1 + n \cdot \log_3 n + n = 1 + n + n \cdot \log_3 n .$$

← أي: $O(n \log_3 n) > O(n)$ ← تعقيد الخوارزمية السابقة $O(n \log_3 n)$.

■ عند وجود المتغير بأكثر من أس أو أكثر من حد ← نأخذ التعقيد الأسوء

تمرين:

1 for(i --> n){	حلقة تتكرر n مرة	n
2 while(j < n)	حلقة تتكرر n مرة	n
3 { j++; }		
4 }		
5 for(i --> n){	حلقة تتكرر n مرة	n
6 while(j < n)	حلقة تتكرر n مرة	n
7 { j++;		
8 for(k --> n)	حلقة تتكرر n مرة	n
9 { ... } }		
10 }		

• نحسب التعقيد:

-ننتبه إلى الحلقات المتداخلة : $T(n) = n \cdot n + n \cdot n \cdot n = n^2 + n^3$

■ عند وجود المتغير بأكثر من أس أو أكثر من حد نأخذ التعقيد الأسوء ← $O(n^3)$.

تمرين:

```

1 for( i=1 ; i<n ; i*=2 ){
2     for( j=0 ; j<n ; j++)
3         { ... }
4 }
5 for( k =0 ; k<n ; k++ ){
6     for( l=0 ; l<n ; l++ )
7         { ... }
8 }

```

$\longrightarrow \log_2 n$
 $\longrightarrow n$
 $\longrightarrow n$
 $\longrightarrow n$

• نحسب التعقيد:

$$T(n) = n \cdot \log_2 n + n \cdot n$$

$$= n^2 + n \log_2 n$$

نأخذ أسوأ تعقيد :

$$O(n^2) > O(n \log_2 n)$$

• $O(n^2) \leftarrow$

تمرين:

شرح:

```

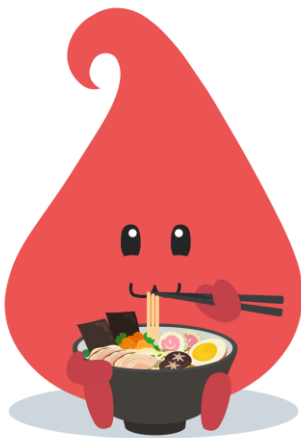
1 int sum = 0;
2 int j = 0;
3 for(int i=0 ; i<n ; i++){
4     for( ; j>i ; j++){
5         sum = sum+10;
6     }

```

نلاحظ هنا أنه عند أول دخول تكون $i=0$ و $j=0$ ، أي أن الشرط $j > i$ غير محقق بالتالي لن ندخل على الحلقة الداخلية، بعدها سيزداد i ليصبح $i=1$ و j سيبقى $j=0$ والشرط $j > i$ غير محقق، حتى تصل i إلى $i=n$ ، أي أن الحلقة الداخلية لن تنفذ طول الكود لأن شرطها لن يتحقق أبداً $\leftarrow$ لن نحسب تعقيدها.

• التعقيد هنا يكون : $T(n) = 1 + 1 + n = 2 + n \rightarrow O(n)$ بفرض كان الشرط في السطر 3 ($j \leq i$) :

عند أول دخول تكون $i=0$ و $j=0$ ، أي أن الشرط $j \leq i$ محقق بالتالي سندخل على الحلقة الداخلية، بعدها سيزداد j ليصبح $j=1$ فالشرط $j \leq i$ سيصبح غير محقق وسنخرج من الحلقة الداخلية وستتكرر هذه العملية من أجل كل قيمة لـ i حتى تصل i إلى $i=n$ ، أي أن الحلقة الداخلية ستنفذ مرة واحدة عند كل i $\leftarrow$ لن تؤثر الحلقة الداخلية على التعقيد.

• التعقيد هنا يكون : $O(n)$.

```

1  bool hasDuplicatedMember (int arr[]) {
2      for (int i = 0 ; i < n ; i ++ ) {
3          for (int j = i+1 ; j < n ; j ++ ) {
4              if (arr[i] == arr[j])
5                  return true ;
6          }
7      }
8      return false;
9  }
10

```

- هنا لا يمكننا ان نقول $O(n^2)$ فوراً لأن الـ for الداخلية لا تتكرر n مرة، لنقوم بحسابها سوياً :

عدد مرات التكرار (الدخول إلى الحلقة)	قيمة j	قيمة i
$n-1$	$j = 1, 2, \rightarrow n-1$	$i = 0$
$n-2$	$j = 2, 3, \rightarrow n-1$	$i = 1$
$n-3$	$j = 3, 4, \rightarrow n-1$	$i = 2$
.	.	.
.	.	.
.	.	.
.	.	.
.	.	.
1	$j = n-1$	$i = n-2$
0	لن ندخل إلى الحلقة الداخلية ($j=n$)	$i = n-1$

- الآن نجمع مرات التكرار :

$$T(n) = 1 + 2 + 3 + \dots + (n-3) + (n-2) + (n-1) = \frac{(n-1)n}{2} = \frac{1}{2}n^2 - \frac{1}{2}n \rightarrow O(n^2) \quad (:$$

```

1  for(int i = 0 ; i < log2(n) ; i ++ ){
2      for(int j = 0 ; j < 2^i ; j ++){
3          .
4          .
5          .
6      }
7  }

```

• لنحسبها معاً (موصفة don't worry 😊) :

عدد مرات التكرار (الدخول إلى الحلقة)	قيمة j	الشرط $j < 2^i$	قيمة i
1	$j = 0$	$j < 2^0 = 1$	$i = 0$
2	$j = 0, 1$	$j < 2^1 = 2$	$i = 1$
4	$j = 0, 1, 2, 3$	$j < 2^2 = 4$	$i = 2$
8	$j = 0, 1, 2, 3, 4, 5, 6, 7$	$j < 2^3 = 8$	$i = 3$
.	.	.	.
.	.	.	.
.	.	.	.
.	.	.	.
.	.	.	.
$n - 1$	$j = 1, 2, 3, \dots, n-1$	$j < 2^{\log_2 n} = n$	$i = \log_2 n$

- الآن نجمع مرات التكرار :

$$T(n) = 1 + 2 + 4 + 8 + \dots + n = 2^0 + 2^1 + 2^2 + 2^3 + \dots + 2^{\log_2 n}$$

- وهي متتالية هندسية أساسها 2 و آخر حد فيها $2^{\log_2 n}$ أي أن مجموعها :

$$T(n) = 2^{\log_2 n + 1} - 1 = 2 * 2^{\log_2 n} - 1 = 2n - 1 \rightarrow O(n)$$

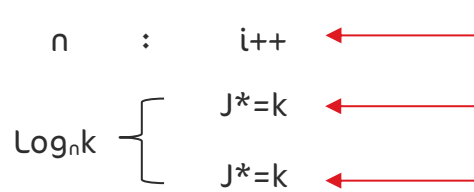
تلخيص أربيسيزي

• تكلفة التعليمات :

- التعريف 0 - التصريح عن تابع 0

- الإسناد 1

- حلقة (for-while) :



" يجب الانتباه إلى القيمة الابتدائية للعداد وتغييره داخل الحلقة و شرط التوقف لأنها ستؤثر على عدد مرات تكرار الحلقة " .

• إذا كانت التعليمات غير متداخلة : نجمع

• إذا كانت التعليمات متداخلة : نضرب

• عند حساب التعقيد : نهمل الثوابت والأمثال ، نأخذ تعقيد الأسوء.

- انتهت المحاضرة -



The sun will always be there

Waiting after the rain 🎵